

1. **Does xv6 kernel use cooperative approach or non-cooperative approach to gain control while a user process is running? Explain how xv6's approach works using xv6's code.**

xv6 mainly uses a non-cooperative (preemptive) scheduling approach to regain control from a running user process. A running process does not need to voluntarily give up the CPU because the kernel can interrupt it periodically using timer interrupts.

In xv6, the timer interrupt is handled inside `trap()` in `trap.c`. When the CPU timer generates an interrupt, the currently running process enters kernel mode automatically. The kernel then checks whether the current process is still running, and if so, it calls `yield()` to force the process to give up the CPU.

In `trap.c`:

```
if(myproc() && myproc()->state == RUNNING &&
    tf->trapno == T_IRQ0+IRQ_TIMER)
    yield();
```

`yield()` function changes the process state from `RUNNING` to `RUNNABLE` and calls `sched()`.

Inside `sched()`, xv6 executes a context switch by calling `swtch()`

```
swtch(&p->context, mycpu()->scheduler);
```

This transfers execution from the current process back to the scheduler.

2. **After `fork()` is called, why does the parent process run before the child process in most of the cases? But in some cases, the child does run first. In what scenario will the child process run before the parent process after `fork()`?**

In xv6, after `fork()` creates a child process, the child process is only marked as `RUNNABLE`. The parent process is still currently running on the CPU, so in most situations the parent continues executing before the scheduler switches to the child.

In `fork()` from `proc.c`, the child process becomes runnable using code:

```
np->state = RUNNABLE;
```

Whereas, the parent process does not immediately stop running after this line. Since the parent already owns the CPU, it usually continues execution until a timer interrupt occurs or the parent voluntarily gives up the CPU. That is why the parent process normally runs first after `fork()`.

Besides, the child process can still run first in some situations. One example is when the parent process gets interrupted by the timer immediately after the child becomes

runnable. In that case, the scheduler may select the child process before the parent resumes execution.

I modified the behavior by adding the following code in `fork()` method, which forces the parent process to voluntarily give up the CPU after creating the child, which allows the child process to run first most of the time.

```
if(fork_winner_policy == 1)
    yield();
```

3. **When the scheduler de-schedules an old process and schedules a new process, it saves the context (i.e., the CPU registers) of the old process and load the context of the new process. Show the code which performs these context saving/loading operations. Show how this piece of code is reached when saving the old process's and loading the new process's context.**

The actual context switching operation in xv6 is performed by the `swtch()` function implemented in `swtch.S`. The scheduler calls `swtch()` inside `sched()` in `proc.c`:

```
swtch(&p->context, mycpu()->scheduler);
```

- `&p->context`: stores the current process context
- `mycpu()->scheduler`: loads the scheduler context

Then, in `scheduler()`, xv6 switches from the scheduler back to a selected process using

```
swtch(&(c->scheduler), p->context);
```

The assembly code in `swtch.S` saves the CPU registers of the old context onto the stack and restores the registers of the new context. Like this:

```
movl %esp, (%eax) - - - - - 1
movl %edx, %esp - - - - - -2
```

The first instruction saves the old stack pointer into the old context structure, while the second instruction loads the new stack pointer from the new context.